

CICLO DE COMPLEMENTACIÓN

LICENCIATURA EN CIENCIA DE DATOS

CLCD 182 - SEMINARIO DE
ACTUALIZACIÓN I

Clases

Semana 03





Temas a Desarrollar

- Tipos de datos básicos
- Secuencias
- Casting
- Mutabilidad
- Colecciones

Booleano (bool)



Tipo de dato que solo admite dos valores, True (verdadero o 1) o False (falso o 0).

Los operadores son: not, and, or.

resultado = not bool_

bool_	resultado
True	False
False	True

resultado = bool_1 and bool_2

bool_1	bool_2	resultado
True	True	True
True	False	False
False	False	False
False	True	False

resultado = bool_1 or bool_2

bool_1	bool_2	resultado
True	True	True
True	False	True
False	False	False
False	True	True



Entero (int)

Tipo de datos que permite representar números enteros, es decir, positivos y negativos no decimales.

Los podemos asignar desde:

- Enteros. Ejemplo: `int(15)` -> **15**
- Decimales. Ejemplo: `int(1.2)` -> **1**
- Binario. Ejemplo: `int(0b100)` -> **4**
- Octal. Ejemplo: `int(0c720)` -> **464**
- Hexadecimal. Ejemplo: `int(0x17)` -> **23**

Límites:

- Mínimo: **-9.223.372.036.854.775.808**
- Máximo: **9.223.372.036.854.775.807**



Decimales (float)

Tipo numérico que permite representar un número positivo o negativo con decimales, Utilizan el estándar IEEE 754 para acelerar los cálculos pero se pierde precisión.

Los podemos asignar como:

- Decimales. Ejemplo: `float(1.2)` -> **1.2**
- Notación científica. Ejemplo: `float(1.93e-3)` -> **0.00193**

Límites:

- Mínimo: `2.2250738585072014e-308`
- Máximo: `1.7976931348623157e+308`



Números Complejos

Tienen dos partes:

- Una parte real, como por ejemplo 3 o 7.5.
- Y otra imaginaria, como 5j o -7j.

Escritura: **parte_real + parte_imaginaria*j** -> $7.5 + 5j$

Son muy utilizados en las telecomunicaciones y electrónica, ya que son muy útiles para describir las ondas electromagnéticas y la corriente eléctrica.

Permiten generar gráficas fácilmente, donde el eje X es la parte real e Y es la parte imaginaria.

Soportan todos los operadores matemáticos.



Cadena de caracteres (str)

Tipo de dato inmutable que permite almacenar secuencias de caracteres.

Definición: comillas dobles " o simples '

Caracteres especiales:

- `\n`: nueva línea
- `\t`: tabulación o 4 espacios en blanco (por defecto)
- `\`: escape para caracteres especiales

Docstring: triple comilla doble `"""`



Cadena de caracteres (str)

Slicing:

```
[inicio:fin:paso]
```

- inicio: es un entero que indica la posición de inicio para la construcción. El valor de inicio es inclusive. Sino se indica el valor de inicio entonces por defecto se comienza desde 0.
- fin: es un entero que indica la posición de fin para la construcción. El valor de fin es no inclusivo. Sino se indica el valor de fin entonces por defecto es la longitud total de la cadena.
- paso: es un entero que indica cuantos pasos hacia adelante o hacia atrás se van a realizar para la construcción. Sino se indica el valor de paso entonces por defecto es 1.



Cadena de caracteres (str)

[inicio:fin:paso]

Letras	a	b	c	d	e
Posiciones	0	1	2	3	4

```
[ ] x:str = "abcde"  
    print(x[0:2])  
⇒ ab
```



Lista (list o typing.List)

Tipo de dato que permite almacenar datos de cualquier tipo y de forma secuencial.

Son secuencias mutables y dinámicas.

Definición:

```
lista:list = [1, 2, 3, 4]
```

Funciones más comunes:

- append
- insert
- extend
- remove
- pop
- clean
- index
- sort
- reverse



Set (set o typing.Set)

Tipo de dato similar a las listas. Los elementos son únicos, desordenados y sus elementos deben ser inmutables.

Definición:

```
s:set = set([5, 4, 6, 8, 8, 1])
```

Funciones más comunes:

- insert
- remove (lanza una excepción)
- discard (eliminar un elemento específico)
- clear
- pop (elimina el primer elemento)



Tupla (tuple o typing.Tuple)

Tipo de dato similar a las listas. Los elementos son inmutables y no se pueden modificar.

Definición:

```
tupla:tuple = (1, 2, 3)
```

Funciones más comunes:

- count
- index



Diccionario o Mapa (dict o typing.Dict)

Tipo de dato que se almacena con el formato clave valor, donde la clave debe ser inmutable.

Definición:

```
d:dict = {'a': 1, 'b': 2}
```

Funciones más comunes:

- keys
- values
- items
- clear
- get
- pop
- popitem (último valor)
- update



Frozenset

Tipo de dato similar a los **set** con la salvedad que los valores se encuentran congelados.

Definición:

```
fs:frozenset = frozenset([1, 2, 3])
```

Funciones más comunes:

- copy
- union
- intersction
- difference



Casting



Conversión entre tipo de datos

- **Implícita:** cuando realizamos ciertas operaciones con dos tipos distintos. (int + float)
- **Explícita:** cuando utilizamos funciones
 - int()
 - float()
 - bool()
 - list()
 - set()



Identidad, tipo y valor

Todo es un objeto y tiene las siguientes propiedades

- **Identidad:** Nunca cambia y es único (**is**).
- **Tipo:** Nos indica el tipo al que pertenece (**type**).
- **Valor:** puede ser
 - **Mutable:** se puede modificar.
 - **Immutable:** no se puede modificar.

Para esto podemos utilizar la función **id**

A horizontal bar with a teal segment on the left and an orange segment on the right.

Identidad, tipo y valor

Mutable: se puede modificar.

- Listas
- Bytearray
- Memoryview
- Diccionarios
- Sets
- Y clases definidas por el usuario



Identidad, tipo y valor

Inmutable: no se puede modificar.

- Booleanos
- Complejos
- Enteros
- Float
- Frozenset
- Cadenas
- Tuplas
- Range
- Bytes



Colecciones

Python viene con un módulo que contiene varios contenedores de datos llamados colecciones o collections en Inglés. (**from collections import**)

- **defaultdict:** no requiere validar las keys
- **OrderedDict:** se ordena según se agregan valores
- **counter:** cuenta el número de elementos
- **deque:** pila o fila de datos (lifo o fifo)
- **namedtuple:** son diccionarios u objetos inmutables
- **enum.Enum (3.4+):** son nombre que pueden tomar un número limitado de estados